

Data Science :HW2

Faezeh Khanmohamadi _810101607

Pooya Anvari _810101384

Parastou Peirovifar_ 810102422

Question 1) Database Design and Analytics

Part 1) Database Design

Your first task is to define a proper database table to house this flat dataset. You must assign optimal data types and identify a primary key.

- Entity Relationship Design: Map the flat columns into at least three logical, distinct tables (e.g., Restaurants, Locations, Services/Metrics). List the columns that belong to each table.
- Keys & Relationships: Clearly identify the Primary Key (PK) for each table. Where tables connect, identify the Foreign Keys (FK) and the type of relationship (1-to-Many, Many-to-Many).
- DDL Execution: Write the SQL CREATE TABLE statements for your designed schema.

۱.۱. طراحی Entity-Relationship (ER Design)

برای اینکه داده‌ها مرتب، قابل مدیریت و بدون تکرار اضافه باشند، فایل خام CSV به یک ساختار رابطه‌ای (Relational Schema) تبدیل شد.

طراحی دیتابیس بر اساس اصول انجام شد تا هم Data Integrity حفظ شود و هم از ذخیره داده‌های تکراری جلوگیری شود.

ساختار نهایی شامل ۳ جدول اصلی است:

جدول locations: این جدول اطلاعات مکانی و جغرافیایی را نگه می‌دارد.

ستون‌ها:

- location_id•
- address•
- neighborhood•
- city•

جدول restaurants : اطلاعات ثابت و هویتی هر شعبه رستوران داخل این جدول ذخیره می‌شود؛ یعنی داده‌هایی که معمولاً تغییر نمی‌کنند.

ستون‌ها:

- restaurant_id•
- name•
- phone•
- rest_type•
- location_id•

جدول services_metrics : این جدول برای داده‌های متغیر و آماری استفاده شد؛ مثل امتیاز، قیمت، منو و نوع سرویس.

ایده اصلی این بود که اطلاعات ثابت رستوران از داده‌های متغیر جدا شوند تا ساختار تمیزتر و نرمال‌تر بماند.

ستون‌ها:

- metric_id•
- online_order•
- book_table•
- rate•
- votes•
- approx_cost•
- listed_in_type•
- cuisines•
- menu_item•
- restaurant_id•

۱.۲ کلیدها و روابط (Keys & Relationships)

Primary Keys.1

برای هر جدول یک کلید اصلی از نوع INT AUTO_INCREMENT تعریف شد:

- locations -> location_id•
- restaurants -> restaurant_id•
- services_metrics -> metric_id•

این کار باعث شد هر رکورد شناسه یکتای خودش را داشته باشد.

2. Foreign Keys و روابط بین جداول

▪ رابطه (One-to-Many) locations → restaurants

در یک منطقه یا آدرس ممکن است چند رستوران مختلف وجود داشته باشد.
به همین دلیل location_id در جدول restaurants به عنوان Foreign Key به جدول locations وصل شد.

▪ رابطه (One-to-Many) restaurants → services_metrics

هر رستوران می تواند چند نوع سرویس یا چند رکورد آماری مختلف داشته باشد؛ مثلاً:

- Delivery
- Dine-out
- Buffet

برای همین restaurant_id در جدول services_metrics به عنوان Foreign Key به جدول restaurants متصل شد.

۱.۳ اجرای DDL

بعد از طراحی ساختار، دستورات SQL مربوط به ساخت دیتابیس و جداول (DDL) اجرا شدند تا Schema نهایی ساخته شود.

۱.۴ منطق انتخاب Data Type ها

▪ INT AUTO_INCREMENT برای Primary Key ها استفاده شد چون:

- سرعت JOIN بالایی دارد
- Indexing سریع تر انجام می شود
- مدیریت شناسه ها راحت تر است

▪ TEXT برای فیلدهای طولانی مثل:

- address
- cuisines
- menu_item

- استفاده شد تا محدودیت طول باعث حذف بخشی از متن نشود.
- VARCHAR برای داده‌های کوتاه‌تر و متغیر مثل:
 - name
 - city
 - rest_type
- استفاده شد چون حافظه را بهینه‌تر مصرف می‌کند و در GROUP BY عملکرد بهتری دارد.
- ENUM('Yes','No') برای وضعیت‌هایی مثل سفارش آنلاین یا رزرو میز استفاده شد تا فقط مقادیر معتبر ذخیره شوند و حجم ذخیره‌سازی هم کمتر شود.
- DECIMAL(3,1) برای فیلد rate استفاده شد تا امتیازها دقیق ذخیره شوند و خطای گرد کردن (Rounding Error) پیش نیاید.
- INT UNSIGNED برای فیلدهای votes و approx_cost استفاده شد چون:
 - مقدار منفی منطقی نیست
 - بازه ذخیره اعداد مثبت بیشتر می‌شود

Part 2) Data Loading

۱. مشکل انکودینگ در زمان بارگذاری (Encoding Fallback)

```
try:
    df = pd.read_csv('zomato.csv', encoding='utf-8')
except UnicodeDecodeError:
    df = pd.read_csv('zomato.csv', encoding='latin1')
```

- مشکلی که حل کرد: فایل‌های حجیم که از وب اسکرپینگ (Web Scraping) به دست می‌آیند، معمولاً دارای کاراکترهای عجیب و غریب هستند که باعث خطای UnicodeDecodeError در پایتون می‌شوند. این بخش با یک استراتژی جایگزین (latin1) تضمین می‌کند که فرآیند خواندن فایل کرش نمی‌کند.

۲. پاکسازی فرمت های عددی

```
df['rate'] = df['rate'].apply(lambda x: float(str(x).split('/')[0]) ...
```

```
df['approx_cost...'] = ...str.replace(',', ' ') ...
df['votes'] = pd.to_numeric(...).fillna(0).astype(int)
```

- مشکلی که حل کرد: در فایل خام، امتیازها به صورت متن "4.1/5" و قیمت‌ها به صورت "1,200" ذخیره شده بودند. پایگاه داده و مدل‌های یادگیری ماشین نمی‌توانند با عبارت "۵/۴.۱" یا کاما در قیمت کار کنند. این بخش اسلش و کاما را حذف می‌کند و آن‌ها را به اعداد واقعی FLOAT و INT تبدیل می‌کند. همچنین آرا (Votes) خالی را به 0 تغییر می‌دهد تا در توابع ریاضی (مثل SUM که در SQL نوشتیم) خطایی تولید نشود.

۳. حذف خطوط اضافه در فایل‌های CSV

```
df[col] = df[col].astype(str).str.replace('\n', ' ').str.replace('\r', ' ')
```

- مشکلی که حل کرد: در بسیاری از رستوران‌ها، آدرس یا لیست منو با دکمه Enter یعنی \n یا \r ثبت شده است. وقتی دستور LOAD DATA INFILE در SQL اجرا می‌شود، هر \n را به عنوان «پایان ردیف» در نظر می‌گیرد! این یعنی اگر یک آدرس دو خطی باشد، SQL خط دوم آدرس را به عنوان یک رستوران جدید در دیتابیس ثبت می‌کند و کل ساختار ستون‌ها به هم می‌ریزد. این کد تمام خطوط شکسته را به فضای خالی (Space) تبدیل می‌کند تا هر رکورد دقیقاً در یک سطر متنی باقی بماند.

۴. کلمه‌ی 'nan'

```
df = df.replace('nan', None)
```

- مشکلی که حل کرد: کتابخانه Pandas در پایتون مقادیر خالی را به NaN تبدیل می‌کند. وقتی این فایل را دوباره به فرمت CSV ذخیره می‌کنید، سلول‌های خالی با کلمه متنی "nan" پر می‌شوند. در نتیجه وقتی داده وارد SQL می‌شود، به جای مقدار استاندارد NULL، کلمه سه حرفی "nan" در دیتابیس ذخیره می‌شود که تمام کوئری‌های تحلیلی را خراب می‌کند. این دستور این تله را خنثی می‌کند.

۵. حذف Ghost Rows و داده‌های بی‌هویت

```
df = df.dropna(how='all')
df = df.dropna(subset=['name', 'address'])
```

- مشکلی که حل کرد: دیتاهایی که کاملاً خالی هستند یا «نام» و «آدرس» ندارند، از نظر منطق پایگاه داده بی ارزش هستند. چون هویت یک رستوران به نام و آدرس آن گره خورده است. وارد کردن رکوردی که آدرس ندارد، قانون یکپارچگی داده‌ها را نقض می‌کند.

۶. کاراکترهای زباله Mojibake Fix

```
cleaned = re.sub(r'^\x00-\x7F]+', '', text)
```

- مشکلی که حل کرد: در فایل خام زوماتو (همان‌طور که در لاگ‌های این کد مشخص است)، آدرس‌هایی وجود دارند که به دلیل باگ‌های سیستمی حاوی کاراکترهای مخدوش و بسیار طولانی (مثل `ÃfÃfÃ,Ãf..`) هستند. این کاراکترهای زباله نه تنها هیچ معنایی ندارند، بلکه طول رشته متنی را به شدت افزایش می‌دهند. در نتیجه، وقتی SQL می‌خواهد این آدرس‌ها را در یک ستون VARCHAR یا TEXT ذخیره کند، ممکن است با خطای طول بیش از حد مواجه شود. این تابع Regex، تمام کاراکترهای غیر انگلیسی (Non-ASCII) را برش می‌دهد و متنی تمیز و کوتاه باقی می‌گذارد.

استفاده از جدول موقت (Staging Table) برای ورود امن داده‌های خام و Ingestion در جداول نرمال‌سازی‌شده.

- ساخت جدول موقت: (zomato_raw) بارگذاری سریع و مستقیم فایل `zomato_cleaned.csv` با استفاده از دستور `LOAD DATA INFILE.`
- استخراج مکان‌ها: (locations) فیلتر و ثبت یکتای مناطق با دستور `SELECT DISTINCT` روی ستون‌های مکانی.
- استخراج هویتی رستوران‌ها: (restaurants) جداسازی موجودیت‌های فیزیکی یکتا (DISTINCT) و اتصال ایمن (JOIN) به جدول مکان‌ها جهت دریافت `location_id`.
- تزریق متریک‌ها و جزئیات سرویس: (services_metrics)
 - انتقال داده‌های متغیر هر سرویس شامل امتیاز، تعداد آرا، منو (menu_item) و نوع غذا. (cuisines)
 - استفاده از عبارات باقاعده (REGEXP) و تبدیل نوع (CAST) برای استخراج و پاکسازی امن داده‌های عددی (حذف ویرگول‌ها و متون اضافه)
 - استفاده از عملگر برابری ایمن (`<=>`) در JOIN پایانی، جهت جلوگیری از ریزش رکوردهای دارای مقادیر NULL مانند شماره‌تلفن‌های ثبت‌نشده.

Part 3) Business Analytics Queries

- تحلیل سرویس دلیوری (Delivery Stats)

برای اینکه تعداد واقعی شعبه‌ها حساب شود و سرویس‌های مختلف باعث شمارش تکراری نشوند، از `COUNT(DISTINCT restaurant_id)` استفاده شد.

همچنین میانگین هزینه سفارش با تابع `AVG(approx_cost)` محاسبه شد تا دید بهتری نسبت به سطح قیمت رستوران‌ها به دست بیاید.

○ خروجی کوئری اول :

	total_delivery_restaurants	average_delivery_cost
▶	8673	433.87

- بررسی تأثیر سفارش آنلاین (Online Order Impact)

با `GROUP BY l.city, s.online_order` رفتار مشتری‌ها در شهرهای مختلف و تفاوت بین رستوران‌هایی که سفارش آنلاین دارند یا ندارند مقایسه شد.

برای جلوگیری از مشکل مقادیر `NULL` در تعداد رأی‌ها هم از `COALESCE` داخل تابع `SUM` استفاده شد تا داده‌ها درست جمع شوند.

○ خروجی کوئری دوم :

	city	online_order	avg_approx_cost	total_votes
▶	Banashankari	No	336.83	19183
	Banashankari	Yes	420.21	82291
	Bannerghat...	No	420.56	29029
	Bannerghat...	Yes	430.61	111558
	Basavanagudi	No	390.64	23810
	Basavanagudi	Yes	450.98	41315
	Bellandur	No	508.81	31742
	Bellandur	Yes	496.31	125780
	Brigade Road	No	822.22	136681
	Brigade Road	Yes	575.66	167818
	Brookefield	No	534.59	39226
	Brookefield	Yes	475.38	118394
	BTM	No	440.02	121206

نتایج نشان می‌دهند که در اکثر شهرها، رستوران‌هایی که سفارش آنلاین دارند: رأی و تعامل بیشتری دریافت می‌کنند و visibility و احتمالاً سهم بازار بالاتری دارند همچنین در بسیاری از مناطق، میانگین قیمت رستوران‌های دارای دلیوری حتی بالاتر است؛ یعنی online delivery فقط مخصوص رستوران‌های ارزان نیست و رستوران‌های premium هم وارد این بازار شده‌اند. البته در بعضی مناطق مثل Brigade Road الگو برعکس است و رستوران‌های بدون دلیوری گران‌تر هستند که احتمالاً به خاطر حضور رستوران‌های لوکس و Fine Dining است که بیشتر روی dine-in تمرکز دارند.

• شناسایی محله‌های برتر (Top-Tier Neighborhoods)

برای پیدا کردن محله‌های باکیفیت‌تر، بعد از گروه‌بندی از HAVING استفاده شد. شرط‌ها این بودند که:

- میانگین امتیاز محله بیشتر از 4.2 باشد.
- حداقل 50 رستوران یکتا در آن محله وجود داشته باشد.
- با انجام این کوئری و این شرایط سختگیرانه هیچ رکوردی وارد تبیل مورد انتخاب ما نشد ،

Result Grid		
Filter Rows:		
neighborhood	restaurant_count	avg_rating

- پس میانگین امتیاز رو از زیاد به کم مرتب کردیم :

	neighborhood	restaurant_count	avg_rating
▶	Whitefield	880	3.60
	Electronic City	729	3.47
	BTM	728	3.56
	HSR	702	3.64
	Marathahalli	681	3.53
	Indiranagar	528	3.78
	JP Nagar	521	3.63
	Bannerghatta Road	474	3.48
	Jayanagar	367	3.76
	Bellandur	361	3.50

نتایج این کوئری نشان داد هیچ محله‌ای به صورت همزمان شرط «میانگین امتیاز بالاتر از ۴.۲» و «حداقل ۵۰ رستوران» را ندارد. بررسی داده‌ها بدون فیلتر HAVING نشان داد محله‌هایی مانند Electronic City ، Whitefield و BTM بیشترین تعداد رستوران را دارند، اما میانگین امتیاز آن‌ها عمدتاً بین ۳.۴ تا ۳.۸ است. این موضوع نشان می‌دهد که محله‌های پرتراکم هنوز به سطح محله‌های ممتاز غذایی (Top-Tier) نرسیده‌اند.

- داشبورد دسته‌بندی قیمتی (Pricing Tier Dashboard)

با استفاده از CASE WHEN رستوران‌ها به سه دسته:

- Budget
- Mid-Range
- Premium

تقسیم شدند تا تحلیل قیمت‌ها راحت‌تر شود.

در انتها هم خروجی با ORDER BY اول بر اساس نام شهر و بعد بر اساس تعداد رستوران‌ها به صورت نزولی مرتب شد تا نمایش داشبورد مرتب‌تر و خواناتر باشد.

- خروجی کوئری چهارم :

city	pricing_tier	restaurant_count	avg_rating
Banashankari	Budget	465	3.62
Banashankari	Mid-Range	116	3.75
Banashankari	Premium	16	3.92
Bannerghatta Road	Budget	826	3.51
Bannerghatta Road	Mid-Range	204	3.57
Bannerghatta Road	Premium	46	3.95
Basavanagudi	Budget	297	3.66
Basavanagudi	Mid-Range	70	3.75
Basavanagudi	Premium	25	3.95
Bellandur	Budget	551	3.52
Bellandur	Mid-Range	173	3.59
Bellandur	Premium	67	4.08

نتایج این کوئری نشان داد که در اکثر شهرها، رستوران‌های دسته Budget بیشترین تعداد را دارند و میانگین امتیاز آن‌ها معمولاً کمتر از دسته‌های گران‌تر است. در مقابل، رستوران‌های Premium تعداد کمتری دارند اما اغلب میانگین امتیاز بالاتری کسب کرده‌اند. این الگو نشان می‌دهد که با افزایش سطح قیمت، کیفیت و رضایت کاربران نیز معمولاً بیشتر می‌شود.

۴. چالش‌های پیاده‌سازی و Resolved Bugs

چالش ۱: تولید شناسه‌های تکراری (Duplicate Entry - Error 1062)
مشکل:

اوایل طراحی، ستون‌های menu_item و cuisines داخل جدول اصلی restaurants بودند. چون بعضی شعبه‌ها برای سرویس‌های مختلف (مثلاً Delivery و Dine-out) منوی متفاوت داشتند، دستور DISTINCT فکر می‌کرد این‌ها چند رستوران جدا هستند و شناسه تکراری تولید می‌شد.

راه‌حل:

ساختار دیتابیس بازطراحی شد و فیلدهای چندمقداری به جدول services_metrics منتقل شدند. این کار باعث شد هر شعبه فقط یک‌بار در جدول اصلی ذخیره شود و ساختار دیتابیس هم به ۳ NF نزدیک‌تر شود.

چالش ۲: شمارش اشتباه در کوئری‌های تحلیلی (Overcounting)
مشکل:

در بعضی Query ها از COUNT(restaurant_id) استفاده شده بود. این موضوع باعث می‌شد به جای تعداد واقعی رستوران‌ها، تعداد سرویس‌ها شمرده شود و آمار داشبورد اشتباه دربیاید.

راه‌حل:

در تمام Query های تحلیلی از COUNT(DISTINCT restaurant_id) استفاده شد تا فقط شعبه‌های یکتا شمارش شوند.

چالش ۳: حذف شدن بعضی داده‌ها هنگام JOIN
مشکل:

بعضی رکوردها شماره تلفن یا نوع رستوران نداشتند (NULL). به خاطر همین، هنگام JOIN بخشی از داده‌ها از خروجی حذف می‌شد چون عملگر = روی NULL درست عمل نمی‌کند.

راه حل:

از عملگر NULL-safe یعنی <=> استفاده شد تا حتی رکوردهایی که مقدار NULL دارند هم درست Join شوند و داده‌ای از بین نرود.

نکته پیاده‌سازی ۱: پاکسازی داده‌ها هنگام ETL

داده‌های خام فرمت تمیزی نداشتند؛ مثلاً امتیازها به شکل "4.1/5" بودند یا هزینه‌ها با ویرگول ذخیره شده بودند. ("1,200") با استفاده از توابعی مثل REGEXP، REPLACE و SUBSTRING_INDEX داده‌ها همان موقع INSERT تمیز و به نوع عددی مناسب تبدیل شدند.

نکته پیاده‌سازی ۲: بهینه‌سازی CASE Statement

برای دسته‌بندی قیمت‌ها، به جای استفاده از شرط‌های سنگین مثل BETWEEN 501 AND 999، از ترتیب اجرای CASE استفاده شد.

یعنی بعد از بررسی بازه کمتر از ۵۰۰، فقط شرط < 1000 چک شد که هم ساده‌تر بود هم پردازش کمتری داشت.

Task 2) Tableau Report

Task 3) Dashboard Improvement using PowerBI

Questions)